

QQUANTT.COM

by mehul.engineer

**RNG Systems Comparative Validation Report:
BBRES-RNG vs Math.random() vs SecureRandom**

Mehul Singh

ID: sys-eng/001/docs

July 14, 2026

Systems Engineering

1. OVERALL VERDICTS AND SUMMARY

A complete battery of 45 tests was executed on 10,918,505 bits and 100,000 integers for each generator.

RNG System	Tests Passed	Failed Tests Count	Status
BBRES-RNG	45/45	0	ARCHITECTURE ACCEPTED
java.security.SecureRandom	43/45	2	REVIEW NEEDED
Java Math.random()	41/45	4	REVIEW NEEDED

While BBRES-RNG achieved a perfect pass rate, SecureRandom showed minor statistical deviations in periodicity and integer distribution (2 failures). Math.random() failed fundamental core bitwise tests (4 failures), indicating it is highly predictable and unsuitable for robust cryptographic entropy.

2. NIST SP 800-22 CORE TESTS COMPARISON (P-VALUES)

The NIST SP 800-22 suite defines the gold standard battery of statistical tests. A p-value below 0.01 indicates a FAIL at the 99% confidence level.

Test	BBRES-RNG	SecureRandom	Math.random()
Frequency (Monobit)	0.441897 (PASS)	0.048856 (PASS)	0.003218 (FAIL)
Block Frequency (M=128)	0.143304 (PASS)	0.110198 (PASS)	0.809584 (PASS)
Cumulative Sums (Fwd)	0.164091 (PASS)	0.088586 (PASS)	0.002812 (FAIL)
Cumulative Sums (Rev)	0.656919 (PASS)	0.068020 (PASS)	0.004243 (FAIL)
Serial (m=2) delta1	0.679895 (PASS)	0.123109 (PASS)	0.007505 (FAIL)
Approximate Entropy	0.851301 (PASS)	0.320284 (PASS)	0.026163 (PASS)

3. SPECTRAL, ENTROPY AND ML ADVERSARIAL ANALYSIS

Spectral analysis checks for periodic components (FFT). ML Adversarial analysis tests if predictive algorithms (Logistic Regression, GBT, or MLP Neural Networks) can predict the next bit in the stream.

Metric/Test	BBRES-RNG	SecureRandom	Math.random()
Spectral FFT (Periodicity)	0.699220 (PASS)	0.003836 (FAIL)	0.102548 (PASS)
Shannon Entropy (8-bit)	7.999860	7.999864	7.999874
ML Attack (GBT, w=16)	0.877536 (Acc: 0.4935)	0.990559 (Acc: 0.4869)	0.097329 (Acc: 0.5072)

Metric/Test	BBRES-RNG	SecureRandom	Math.random()
SHA-256 CSPRNG Wrapper	1.000000 (PASS)	1.000000 (PASS)	1.000000 (PASS)

4. INTEGER DISTRIBUTION AND SEQUENCE COMPARISON

Evaluating integers generated uniformly in the range [0..999] across 100,000 samples.

Test	BBRES-RNG	SecureRandom	Math.random()
Anderson-Darling	0.357194 (PASS)	0.000520 (FAIL)	0.219754 (PASS)
Chi-Square Uniformity	0.475159 (PASS)	0.799222 (PASS)	0.145997 (PASS)
Lag-1 Autocorrelation	0.418097 (PASS)	0.612490 (PASS)	0.861625 (PASS)
Median Test	0.839534 (PASS)	0.819826 (PASS)	0.844479 (PASS)

Conclusion: The BBRES-RNG scheduler-based methodology effectively harvests non-deterministic OS entropy to deliver completely uncompromised randomness, objectively matching and exceeding the statistical quality of Java's cryptographic `SecureRandom`, while utterly outperforming the flawed Linear Congruential generator underlying `Math.random()`.

REFERENCES

BBRES-RNG Combined Validation Report: [https://qquantt.com/files/sys-eng/001/docs/Reports/BBRES-RNG Combined Validation Report.pdf](https://qquantt.com/files/sys-eng/001/docs/Reports/BBRES-RNG%20Combined%20Validation%20Report.pdf)
 Java Math.random() Combined Validation Report: [https://qquantt.com/files/sys-eng/001/docs/Reports/Java Math.random\(\) Combined Validation Report.pdf](https://qquantt.com/files/sys-eng/001/docs/Reports/Java%20Math.random()%20Combined%20Validation%20Report.pdf)
 java.security.SecureRandom Combined Validation Report: [https://qquantt.com/files/sys-eng/001/docs/Reports/java.security.SecureRandom Combined Validation Report.pdf](https://qquantt.com/files/sys-eng/001/docs/Reports/java.security.SecureRandom%20Combined%20Validation%20Report.pdf)
 View Source Code for BBRES-RNG: <https://github.com/mehul-engineer/Bit-Based-Randomized-Entropy-System-Scheduler-Based-Entropy-System>